

MedFlow - Présentation professionnelle de défense

Support amélioré pour une soutenance de Licence en Informatique, à partir de la documentation technique du projet MedFlow.

Slide 1 - Titre académique

Conception et réalisation de MedFlow

Application web de gestion électronique des dossiers patients

Projet présenté en vue de l'obtention du diplôme de **Licence en Informatique**.

- Domaine : informatique de gestion appliquée au secteur hospitalier
- Projet : MedFlow, système de gestion hospitalière orienté dossier patient
- Contexte : établissements de santé à Kinshasa, RDC
- Stack : Next.js 15, TypeScript, PostgreSQL/Neon, Prisma, Clerk, Pusher

Notes orales Mesdames et Messieurs les membres du jury, je vous présente MedFlow, un projet de conception et réalisation d'une application web destinée à améliorer la gestion des dossiers patients et des processus hospitaliers associés. Ce travail s'inscrit dans le domaine de l'informatique de gestion, avec une application concrète au secteur de la santé.

Slide 2 - Plan de présentation

Fil conducteur de la défense

1. Contexte et problématique
2. Objectifs du projet
3. Méthodologie de conception et de réalisation
4. Analyse fonctionnelle et acteurs du système
5. Architecture technique
6. Modèle de données
7. Modules réalisés
8. Sécurité, validation et traçabilité
9. Démonstration proposée
10. Résultats, limites et perspectives

Notes orales Je vais d'abord présenter le problème auquel le projet répond, puis les objectifs et la méthode utilisée. Ensuite, j'expliquerai la conception technique, les principaux modules réalisés, les mécanismes de sécurité, avant de terminer par les résultats, les limites et les perspectives.

Slide 3 - Contexte général

La numérisation comme levier d'amélioration du parcours patient

Dans beaucoup de structures de santé, plusieurs opérations restent fragmentées : accueil du patient, suivi des rendez-vous, consultation médicale, facturation, encaissement et archivage.

Cette fragmentation peut entraîner :

- une perte de temps dans la recherche d'informations ;
- des erreurs dans le suivi du patient ;
- une faible visibilité sur les paiements ;
- une traçabilité insuffisante des actions sensibles ;
- une coordination difficile entre services.

Notes orales Le contexte du projet part d'un constat simple : les informations liées au patient circulent souvent entre plusieurs services sans être réellement centralisées. Or, dans un établissement de santé, la continuité de l'information est essentielle pour la qualité du service, la rapidité de prise en charge et la fiabilité administrative.

Slide 4 - Problématique

Comment centraliser et sécuriser le parcours patient ?

Question centrale :

Comment concevoir une application web capable de centraliser les informations médicales, administratives et financières d'un patient, tout en garantissant un accès sécurisé selon les responsabilités de chaque utilisateur ?

Contraintes principales :

- gérer plusieurs profils d'utilisateurs ;
- préserver la cohérence des données médicales ;
- relier rendez-vous, consultation, dossier et facture ;
- assurer la traçabilité ;
- adapter le système au contexte local, notamment les modes de paiement.

Notes orales La problématique n'est pas seulement de stocker des données. Il faut organiser un flux complet, depuis l'enregistrement du patient jusqu'au paiement, avec des rôles clairement définis et des données cohérentes.

Slide 5 - Objectif général

Mettre en place un système web intégré de gestion hospitalière

L'objectif général de MedFlow est de concevoir et réaliser une application web permettant de centraliser le parcours patient dans un établissement de santé.

Le système doit permettre de :

- enregistrer et gérer les patients ;
- planifier les rendez-vous ;
- suivre les consultations ;
- gérer les dossiers médicaux ;
- facturer les prestations ;
- encaisser les paiements ;
- notifier les utilisateurs concernés ;

- conserver un historique des actions sensibles.

Notes orales L'objectif général est donc de proposer une solution intégrée. Le mot important ici est intégré : les modules ne sont pas indépendants, ils sont reliés autour du patient et du rendez-vous.

Slide 6 - Objectifs spécifiques

Transformer les besoins métier en fonctionnalités mesurables

Objectifs fonctionnels :

- créer des dossiers patients complets ;
- organiser les rendez-vous selon les médecins et les créneaux ;
- enregistrer les signes vitaux, diagnostics et prescriptions ;
- générer des factures avec reçu unique ;
- gérer des sessions de caisse ;
- envoyer des notifications en temps réel.

Objectifs techniques :

- utiliser une architecture modulaire ;
- sécuriser l'accès par rôle ;
- valider les données côté serveur ;
- structurer la base relationnelle ;
- produire une interface responsive et maintenable.

Notes orales Ces objectifs montrent le passage du besoin métier vers des fonctionnalités concrètes. Ils permettent aussi d'évaluer le projet : chaque objectif correspond à un module ou à une contrainte technique vérifiable.

Slide 7 - Méthodologie

Une démarche orientée analyse, conception et réalisation

La réalisation a suivi une démarche progressive :

1. Analyse du domaine hospitalier et identification des acteurs.
2. Définition des modules fonctionnels prioritaires.
3. Conception de l'architecture applicative.
4. Modélisation de la base de données avec Prisma.
5. Développement des interfaces, actions serveur et API routes.
6. Intégration de l'authentification, des permissions et des notifications.
7. Vérification des flux principaux : patient, rendez-vous, consultation, caisse.

Notes orales J'ai adopté une démarche structurée, proche d'un cycle incrémental. L'idée était de construire d'abord le socle : acteurs, données et architecture, puis d'ajouter progressivement les modules métier.

Slide 8 - Acteurs du système

Un système multi-profils avec responsabilités séparées

Acteur	Responsabilités principales
Admin	Gestion des utilisateurs, paramètres, audit, supervision
Médecin	Consultation, diagnostic, dossier médical
Infirmier	Signes vitaux, suivi patient, administration médicale
Patient	Profil, rendez-vous, ordonnances, paiements
Caissier	Sessions de caisse et encaissements
Technicien labo	Examens de laboratoire selon le périmètre

Chaque acteur accède à un espace adapté à son rôle.

Notes orales La séparation des rôles est essentielle. Elle répond à un besoin d'ergonomie, mais aussi à un besoin de sécurité. Un caissier n'a pas les mêmes responsabilités qu'un médecin, et un patient ne doit pas accéder aux informations administratives globales.

Slide 9 - Cas d'utilisation principaux

Les scénarios métier couverts par MedFlow

Principaux cas d'utilisation :

- s'authentifier et accéder à son tableau de bord ;
- enregistrer un patient ;
- créer et confirmer un rendez-vous ;
- consulter les détails d'un rendez-vous ;
- enregistrer les signes vitaux ;
- ajouter un diagnostic et des prescriptions ;
- générer une facture ;
- encaisser un paiement ;
- consulter les notifications ;
- auditer les opérations sensibles.

Notes orales Ces cas d'utilisation représentent le cœur du système. Ils suivent la réalité d'un parcours patient : accueil, rendez-vous, consultation, facturation et suivi.

Slide 10 - Choix technologiques

Une stack moderne pour une application full-stack typée

Couche	Technologies retenues
Framework	Next.js 15 avec App Router
Langage	TypeScript
Interface	React 18, Tailwind CSS, shadcn/ui, Radix UI
Base de données	PostgreSQL via Neon

Couche	Technologies retenues
ORM	Prisma 6
Authentification	Clerk
Temps réel	Pusher
Validation	Zod et React Hook Form
PDF	@react-pdf/renderer
Graphiques	Recharts

Notes orales Le choix de Next.js permet d'avoir dans un même projet le frontend, les pages serveur, les API routes et les server actions. TypeScript et Prisma apportent un typage fort, ce qui réduit les erreurs dans une application où les données sont sensibles.

Slide 11 - Architecture globale

Une architecture en couches pour séparer les responsabilités

```
Navigateur utilisateur
  -> composants React et formulaires
Next.js App Router
  -> pages, layouts, API routes, server actions
Couche métier
  -> lib/* et utils/services/*
Prisma
  -> client ORM et adaptateur Neon
PostgreSQL
  -> persistance relationnelle
```

Cette organisation limite le couplage entre interface, logique métier et accès aux données.

Notes orales L'architecture est organisée en couches. Les composants affichent les données, les server actions et API traitent les opérations, les services manipulent les requêtes métier, et Prisma communique avec PostgreSQL. Cette séparation facilite la maintenance.

Slide 12 - Organisation du projet

Un découpage cohérent avec l'architecture Next.js

- [app/](#) : routes, pages protégées, API routes et server actions.
- [components/](#) : composants UI, formulaires, tableaux, caisse, rendez-vous, PDF.
- [lib/](#) : base de données, permissions, audit, notifications, i18n, Pusher.
- [utils/services/](#) : couche d'accès aux données par domaine métier.
- [prisma/](#) : schéma de données, migrations et seed.
- [docs/](#) : documentation technique et support de présentation.

Le projet distingue clairement les responsabilités techniques.

Notes orales La structure du projet est importante dans une défense, car elle montre la capacité à organiser une application qui peut évoluer. Chaque dossier a une responsabilité identifiable.

Slide 13 - Modèle de données

Un modèle relationnel centré sur le patient et le rendez-vous

Le schéma Prisma contient **19 modèles** et **13 enums**.

Entités principales :

- **Patient** : identité, contact, consentements, assurance, antécédents ;
- **Doctor** : spécialisation, licence, disponibilité, horaires ;
- **Appointment** : lien entre patient, médecin, date, heure et statut ;
- **MedicalRecords** : dossier médical rattaché au rendez-vous ;
- **Diagnosis, VitalSigns, LabTest** : informations cliniques ;
- **Payment, PatientBills, Services** : facturation ;
- **CashierSession** : suivi des encaissements ;
- **Notification** et **AuditLog** : communication et traçabilité.

Notes orales Le choix d'un modèle relationnel est justifié par la nature fortement liée des informations hospitalières. Le rendez-vous devient un point d'articulation entre le patient, le médecin, le dossier médical et la facture.

Slide 14 - Intégrité des données

Des contraintes pour protéger les règles métier

Règles importantes intégrées au modèle :

- un médecin ne peut pas avoir deux rendez-vous au même créneau ;
- un patient ne peut pas avoir deux rendez-vous au même créneau ;
- une facture finale est liée à un rendez-vous unique ;
- chaque reçu possède un numéro unique ;
- les champs fréquemment recherchés sont indexés ;
- plusieurs relations utilisent des suppressions en cascade maîtrisées.

Ces contraintes évitent une partie des incohérences directement au niveau de la base.

Notes orales Ces règles ne sont pas de simples détails techniques. Elles traduisent des règles métier : éviter les conflits d'agenda, empêcher les doublons de reçu et garantir que les informations restent liées correctement.

Slide 15 - Authentification et autorisation

Un contrôle d'accès basé sur les rôles

Flux de sécurité :

Requête HTTP

- > middleware.ts
- > vérification de session par Clerk
- > lecture du rôle dans les métadonnées JWT
- > comparaison avec lib/routes.ts
- > accès autorisé ou redirection

Exemples :

- `/admin(.*)` : admin uniquement ;
- `/cashier(.*)` : cashier ou admin ;
- `/record/patients` : admin, doctor, nurse ;
- `/nurse(.*)` : nurse ou admin.

Notes orales La sécurité est traitée avant même l’affichage des pages grâce au middleware. Le rôle de l’utilisateur est fourni par Clerk puis comparé aux routes autorisées. Les helpers serveur renforcent cette protection pour les actions sensibles.

Slide 16 - Module patient

Centraliser l’identité et l’historique médical du patient

Fonctionnalités principales :

- inscription du patient ;
- informations personnelles et contacts d’urgence ;
- consentements : confidentialité, service et suivi médical ;
- antécédents, allergies, groupe sanguin ;
- assurance ;
- recherche, pagination et consultation de profil ;
- lien avec rendez-vous, diagnostics, paiements et évaluations.

Notes orales Le patient est l’entité centrale. Le dossier patient rassemble les informations nécessaires à l’accueil, à la consultation et au suivi administratif.

Slide 17 - Module rendez-vous

Organiser le parcours clinique autour du rendez-vous

Workflow principal :

Création de la demande

-> PENDING

Confirmation du rendez-vous

-> SCHEDULED

Consultation

-> signes vitaux + diagnostic + dossier médical

Fin de consultation

-> COMPLETED
Annulation possible
-> CANCELLED

Le rendez-vous relie le patient, le médecin, la consultation et la facture.

Notes orales Le rendez-vous est l'un des éléments les plus importants du système. Il donne un cadre temporel et métier à la consultation. Il sert aussi de point de départ pour la facturation.

Slide 18 - Module dossier médical

Structurer les informations cliniques de la consultation

Le dossier médical gère :

- signes vitaux : température, tension, fréquence cardiaque, SpO2, poids, taille ;
- diagnostic : symptômes, conclusion, notes ;
- prescriptions et plan de traitement ;
- demandes d'examens laboratoire ;
- historique rattaché au patient, au médecin et au rendez-vous.

Chaque donnée clinique est rattachée à un contexte précis.

Notes orales Le dossier médical n'est pas une fiche isolée. Il est lié à une consultation précise, ce qui permet de savoir quand l'information a été produite et par quel professionnel.

Slide 19 - Module facturation

Relier les prestations médicales au paiement

Fonctionnalités :

- configuration des services facturables ;
- ajout de lignes de facture par prestation ;
- calcul du montant total ;
- gestion des remises ;
- paiements partiels ou complets ;
- génération d'un reçu unique ;
- export PDF de la facture.

La facture est liée au rendez-vous, donc au dossier patient.

Notes orales La facturation n'est pas traitée comme un module séparé du soin. Elle est reliée au rendez-vous, ce qui permet de suivre les actes facturés et le statut du paiement.

Slide 20 - Module caisse

Contrôler les encaissements par session de caisse

Workflow caisse :


```
Ouverture de session
-> montant initial
Encaissement des paiements impayés
-> rattachement à la session
Clôture de session
-> totaux par mode de paiement
-> statut CLOSED
```

Modes adaptés au contexte local : espèces CDF/USD, carte, Mobile Money, assurance et virement bancaire.

Notes orales La session de caisse permet de responsabiliser le caissier et de produire un contrôle de fin de période. Les modes de paiement tiennent compte du contexte local, notamment Mobile Money.

Slide 21 - Notifications temps réel

Informer rapidement sans perdre l'historique

Architecture des notifications :

```
Action métier
-> création d'une notification en base
-> envoi Pusher sur canal privé
-> réception dans NotificationBell
-> lecture, suppression ou marquage via API
```

Types : nouveau rendez-vous, confirmation, annulation, modification, paiement reçu et information générale.

Notes orales Les notifications sont d'abord enregistrées en base, puis envoyées en temps réel. Ainsi, même si l'utilisateur n'est pas connecté au moment de l'événement, il peut retrouver l'information plus tard.

Slide 22 - Validation et robustesse

Réduire les erreurs par plusieurs niveaux de contrôle

Mécanismes utilisés :

- formulaires structurés avec React Hook Form ;
- validation des données avec Zod ;
- contraintes Prisma et PostgreSQL ;
- index sur les champs de recherche ;
- migrations versionnées ;
- seed pour initialiser les services et modes de paiement ;
- journalisation des actions sensibles dans **AuditLog**.

Notes orales La robustesse ne repose pas sur une seule couche. Les données sont contrôlées à l'entrée, structurées dans le modèle, puis protégées par les contraintes de la base. L'audit complète ce dispositif.

Slide 23 - Interface utilisateur

Une expérience adaptée aux profils utilisateurs

Principes d'interface :

- layout protégé avec sidebar et navbar ;
- navigation dynamique selon le rôle ;
- composants shadcn/ui pour cohérence et accessibilité ;
- tableaux, modales, formulaires, graphiques ;
- mode clair/sombre ;
- internationalisation français/anglais ;
- interface responsive pour desktop et mobile.

Notes orales L'interface a été pensée pour que chaque utilisateur accède rapidement à ses tâches. La navigation par rôle réduit la surcharge visuelle et rend l'application plus simple à utiliser.

Slide 24 - Démonstration proposée

Présenter le projet à travers un parcours patient complet

Scénario conseillé devant le jury :

1. Connexion avec un rôle autorisé.
2. Affichage du tableau de bord et de la navigation filtrée.
3. Création ou consultation d'un patient.
4. Création d'un rendez-vous.
5. Consultation : signes vitaux, diagnostic, dossier médical.
6. Génération de facture.
7. Ouverture de session caisse et encaissement.
8. Notification temps réel.
9. Consultation de l'audit ou des paramètres admin.

Notes orales La démonstration doit raconter une histoire simple : un patient arrive, il est enregistré, il obtient un rendez-vous, il est consulté, puis il passe à la caisse. Ce scénario montre la cohérence de l'ensemble du système.

Slide 25 - Résultats obtenus

Une application complète couvrant les flux essentiels

Résultats fonctionnels :

- gestion des profils et des rôles ;
- gestion des patients, médecins, staff et rendez-vous ;
- dossier médical lié à la consultation ;
- facturation et encaissement ;
- notifications temps réel ;
- audit et paramètres d'administration.

Résultats techniques :

- architecture Next.js modulaire ;
- modèle Prisma structuré ;
- contrôle d'accès par middleware ;
- intégration PostgreSQL, Clerk et Pusher.

Notes orales Le résultat principal est un socle applicatif fonctionnel et cohérent. Le projet ne se limite pas à une interface : il intègre l'authentification, la base de données, les workflows métier et le temps réel.

Slide 26 - Limites du projet

Une analyse critique nécessaire pour une soutenance solide

Limites actuelles :

- absence de suite complète de tests automatisés ;
- module laboratoire encore améliorable ;
- permissions médicales pouvant être rendues plus fines ;
- monitoring et sauvegardes à formaliser ;
- conformité réglementaire santé à approfondir selon l'établissement ;
- indicateurs analytiques encore limités.

Notes orales Reconnaître les limites ne diminue pas la valeur du projet. Au contraire, cela montre une capacité d'analyse critique et une vision réaliste de ce qu'il faudrait faire pour passer vers une solution de production complète.

Slide 27 - Perspectives

Évolutions possibles après la défense

Améliorations futures :

- tests unitaires et end-to-end ;
- module pharmacie et gestion des stocks ;
- module laboratoire plus complet ;
- rapports statistiques et exports administratifs ;
- messagerie interne ;
- sauvegardes automatiques et monitoring ;
- audit de sécurité ;
- règles de conformité et confidentialité plus détaillées.

Notes orales Les perspectives montrent que MedFlow peut évoluer vers un système d'information hospitalier plus large. Les priorités futures seraient les tests, la sécurité, la conformité et l'enrichissement des modules métier.

Slide 28 - Questions probables

Réponses techniques à préparer

Pourquoi Next.js ?

Pour disposer d'un framework full-stack combinant interface, rendu serveur, API routes et server actions.

Pourquoi Prisma ?

Pour modéliser clairement les relations, gérer les migrations et bénéficier du typage.

Comment les accès sont-ils sécurisés ?

Par Clerk, le middleware RBAC, les helpers de permissions et la séparation des routes.

Comment éviter les conflits de rendez-vous ?

Par des contraintes uniques sur le médecin, le patient, la date et l'heure.

Pourquoi Pusher ?

Pour informer les utilisateurs en temps réel sans recourir au polling permanent.

Notes orales Cette slide est une aide à la préparation. Les réponses doivent rester courtes, techniques et liées aux besoins métier.

Slide 29 - Conclusion

MedFlow, une solution intégrée pour le parcours patient

MedFlow propose une base complète pour la gestion électronique des dossiers patients :

- centralisation des informations médicales et administratives ;
- coordination entre patient, médecin, infirmier, administration et caisse ;
- sécurité par rôle ;
- traçabilité des actions sensibles ;
- architecture web moderne et maintenable ;
- adaptation au contexte local.

Phrase de clôture : MedFlow illustre comment l'informatique peut améliorer l'organisation, la fiabilité et la continuité de l'information dans un établissement de santé.

Notes orales Pour conclure, je rappellerais que la valeur du projet réside dans son intégration. MedFlow relie les modules essentiels du parcours patient et montre une application concrète des compétences acquises en licence informatique.